CSCU9P6 Assessment: Group Project (70% of module mark)

For the purpose of this project each of you is required to operate as a member of a team. The task of the team is to implement a working system in Java satisfying the object-oriented design described later in this document.

Lists of team members and monitoring officers, this description and all project materials are on Canvas on the Unit 7 page in "Learning and Teaching" or on your "Groups" page.

Sommerville's and Pressman's texts on software engineering provide guidance on organizing teamwork, alongside the material presented in the module's software project management seminars. Each group **must** appoint one of its members as *Project Manager*. The Project Manager is responsible for planning, tracking and updating the work schedule using a *Gantt chart, kanban board* and/or other scheduling tools.

Individual Reflective Diary

Each group member must keep their own reflective diary, in a Microsoft Word file. This must be stored on your University OneDrive, so that changes are tracked, in accordance with the AI coversheet.

Your diary must have one entry for each week (300-500 words per entry). The entry for each week should *reflect* on the work that you have done that week, explaining the context, what you have achieved, and what you have learnt from doing that. To do so, you must STAR-L format for each week's entry, as follows (see https://research.qut.edu.au/eportfolio/students-alumni/reflective-practises-writing-make-a-start-and-maintain-the-momentum/):

Situation: Briefly explain the context of the main task you were working on -- how does it fit into the wider project and the project plan? Were you working with any other group members?

Task: What was the specific task you were required on? How complex was it and what made it challenging?

Action: What did you do? This must be written from an individual perspective. Provide a high-level summary in the diary entry. Support this with metrics such as the time spent on different parts of the task and/or the (approximate) number of lines of code or tests written. Provide links (URLs) to the related issue and commits on GitHub. The issues and commits form the evidence that you have done this.

Results: What was the actual outcome achieved by doing this? As a result of your actions, which bits of functionality were actually implemented, or which bugs were actually fixed?

Learning: What did you learn from doing this task? Which aspects do you want to follow-up on next week? What would you do differently if you did it again? How might what you have learnt apply to other tasks either in this project or in the future?

Group Meetings

For each team we will appoint a *project monitoring officer*. The monitoring officer will meet weekly with your group beginning after reading week. Each group member **must** attend these weekly meetings. Attendance and engagement at these meetings forms part of the evidence that you have worked on the project, corroborates your individual reflective diary and assessment of individual contributions form, and demonstrates your professionalism as a software engineer. **You must attend a minimum of two of the four timetabled group meetings, otherwise the mark for your individual reflective diary will be capped at 40%.** Of course, you should aim to attend all meetings in order to assist and get help from your fellow group members.

Each meeting will last 10-15 minutes (you will be allocated an exact time within the two-hour "workshop" slot on your timetable). Most meetings will take the form of a stand-up, as used in Agile Scrum, see: https://www.atlassian.com/agile/scrum/standups. During the stand-up meeting, each group member in turn will answer three questions:

1. What have I accomplished since the last meeting?
2. What will I do before the next meeting?
3. Are there any blockers?

The purpose of this is to allow the group to discuss progress and identify any blockers (e.g. maybe you don't know how to do something in Java, once this is known, another group member can help you after the meeting).

In addition to these weekly mandatory meetings with your monitoring officer, your group should have at least one other stand-up meeting per week to ensure consistent progress. Remember, **each group member should be working on the project for approximately 12 hours per week**. These additional stand-ups could take place online or in-person, and should be organized by the Project Manager.

Formative Feedback (interim checkpoint)

You must submit an interim checkpoint to Canvas by Friday 20th March. This checkpoint does not count towards your final mark, but is used to inform the monitoring officer of the group's organization and progress and to provide feedback on this.

The interim checkpoint submission consists of your project plan (e.g. Gantt chart or Kanban board) showing the work completed so far and the work still to be completed. This should clearly show which tasks have been allocated to each group member and the progress of each. You should also upload your current version of the class diagram.

Feedback will take the form of a discussion with your monitoring office during the week beginning 23rd March (this will take the place of your stand-up meeting during that week).

Use of GitHub

It is **mandatory** for your group to use GitHub, following the GitHub flow workflow covered in Lecture 5, and Practical 5.

The Project Manager should create a repository (repo) for the group. They should add the other group members as collaborators. The module coordinator should also be added as a collaborator (Simon's GitHub username is simon-t-powers).

As a reminder, the GitHub flow workflow covered in Unit 5 is as follows (and is common in industry). Each task to be done should have an issue created on GitHub, which should be assigned to the group member who will complete this task. A branch should be created for the issue, and development for that task should take place on that branch. Regular commits should be made while working on the issue. Each commit should have a meaningful message. Commits should be regularly pushed to GitHub, to keep your work synchronised with the group's repository. When the task is fully completed, and the final version pushed to GitHub, the group member who has completed the task should create a pull request. They should assign this to another group member to code review (the Project Manager should create a policy for who review's whose code, and the group should agree rules on what a code review involves, see e.g. https://github.blog/developer-skills/github/how-to-review-code-effectively-a-github-staff-engineers-philosophy/). After addressing any comments satisfactorily, the pull request should be merged back into the main branch.

This video on Canvas gives a complete walkthrough of this process in NetBeans: https://stirling.cloud.panopto.eu/Panopto/Pages/Viewer.aspx?id=67393cef-e1ec-4c4d-a2e5-b3f5009f293b

Remember, when using GitHub you won't see changes from other group members unless you do a pull operation (on the Teams menu in NetBeans). You should make sure to do a pull at the start of each work session and a push at the end. You should also only be working on

the branch for your current issue. You should not directly work on the main branch – only code that has been reviewed as a pull request should be merged onto main. This minimises the risks of conflicts given that several group members will be coding at the same time.

Important: You make your GitHub repository accessible to your group members and the module coordinator only, and ensure that it remains private after the project is over. If your repository is public, your work will be exposed to plagiarism (it would be visible to current and future students at Stirling and at partner institutions abroad where this module is also taught) and the integrity of the assignment will be compromised.

What to submit

Each group will make a **group submission** as detailed below.

In addition, each individual student will submit their own **Individual Reflective Diary**, and their **Assessment of Individual Contributions Form**.

Group submission

Each group will submit the following components to Canvas:

- The final Java files, **submitted** as a single Zip file.
- **A Screencast video** of the working system. The video must demonstrate as fully as possible the working of each of the use cases given in the use case diagram in the project specification (make sure to check the written use case descriptions attached to each use case carefully – these form the requirements of the system). Your submission must be an MP4 file recording that is no more than 30 minutes long. You should plan the sequence of actions that you will take so that you can demonstrate all of the use cases in this time.
  It is recommended that the Project Manager demonstrates the system. However, this task can be delegated to another group member. To ensure a smooth demonstration, it is recommended that only one group member takes part in the recording.
- **A report** on *JUnit testing* to demonstrate the correctness of your implementations of the `Lobby`, `MeetingSession`, `Reservation` and `VirtualRoom` classes. For simplicity assume that the classes that they depend on are themselves all correct and so may be used in the testing. The documentation must include the full code of the JUnit tests, and a description of the **rationale** for the test cases chosen (well written comments in the JUnit test code would be fine). Show also the results of applying the tests.

If your group did not fully implement the `Lobby`, `MeetingSession`, `Reservation` and `VirtualRoom` classes your report should still include explanations and code for the tests that you would use for a completed implementation.

*You may (should!) wish to carry out unit testing of other classes too, but you should **not** hand in details of those tests.*

In addition, each group **must attend a 20-minute demonstration at their scheduled time** (which will be announced on Canvas). During the demonstration you will be asked questions about your design and code and asked to justify decisions. You will also be asked about your group's working process (e.g. use of GitHub issues, code reviews...). Every group member must attend the demonstration. Group members that do not attend will receive a mark of 0 for the demonstration.

Individual submission

You must submit your **Individual Reflective Diary**. In addition, you must submit an **Assessment of Individual Contributions Form** (available on Canvas Unit 7). The assessments of individual contributions are used to adjust the mark for the group components (Screencast Video, JUnit Test Report and Demonstration) based on the forms submitted by their entire group and on the observations of the group monitoring officer.

Overall mark weighting

The overall mark for this assessment is weighted as follows:

1. Evidence of implementation of use cases from Screencast Video: 50%
2. JUnit Test Report: 15%
3. In-person Demonstration of design, code and group working process: 15%
4. Individual Reflective Diary: 20%

The marks for 1., 2. and 3. will be adjusted for each group according to the information on the Assessment of Individual Contributions forms from the members of that group (see assignments on Canvas for details of this calculation).

Marking rubrics for each element of the submission are provided on Canvas.

Note on Independent Working and Avoiding Plagiarism

Work which is submitted for assessment must be your own work. All students should note that the University has a formal policy on Academic Integrity and Academic Misconduct (including plagiarism) which can be found at https://www.stir.ac.uk/media/stirling/services/academic-registry/documents/Academic-Integrity-Policy.pdf

To avoid the risk of your own work being plagiarised by others, keep your repository private both during and after the assignment period.

Note also that asking or paying someone else to do assignment work for you (contract cheating) is considered gross academic misconduct, and will result in termination of your studies with no award.

Use of Artificial Intelligence

For this assessment, AI Planning in the AI in Assessments Scale is permitted. This means that AI can be used in the assessment for planning, idea development, and research. Where AI is used in this way it must be acknowledged (e.g. with comments in the code acknowledging where AI has influenced the design of that code). **However, no AI generated content is allowed in the final submission – all code must be written by yourself. No AI generated text or images are permitted in your JUnit Test Report or Individual Reflective Diary.**

Late submission and extensions

The standard University policies for dealing with late submissions apply. Assessed coursework submitted late will be accepted up to seven calendar days after the deadline date but the mark will be lowered by three marks per day or part thereof. After seven days the piece of work will be deemed a **non-submission**; this will result in the award of grade X for the module.

Due to the nature of group work, **extensions will not be provided** for the group submission (Screencast Video, JUnit Report, and Demonstration).

Late submission of any part of the group submission will result in penalties for all members of the group.

Late submission of a group member's Individual Reflective Diary or Assessment of Individual Contributions Form will result in penalties for that group member only.

The standard extension process applies to the Individual Reflective Diary and Assessment of Individual Contributions Form. Late submission of this outside of an extension will result in standard late submission penalties for the Individual Reflective Diary mark.

Virtual Career Fair System (VCFS) Project Specification

Introduction

For this assignment you will be implementing a system to be used for running virtual career fairs: the Virtual Career Fair System (VCFS). A virtual career fair is a one-day event that has been set aside for candidates to meet recruiters from different organizations, find out about opportunities, and have short appointment-style meetings online. The VCFABS system will allow candidates to make reservations for appointments with recruiters (via the organizations' booths) and will also provide a video-conferencing style service to allow live sessions to take place online.

The real system would be implemented as an internet service, with different browser or desktop application interfaces for recruiters, candidates, and admin staff, a central database, and real video-conferencing capability. The assignment is a simplified exercise, so the devices will be simulated and all the interfaces will appear as separate windows on a single PC screen. There won't be any persistent storage of data (so no database is needed), and the video-conferencing will be simulated, perhaps by just showing windows containing the names of the people who are meeting each other (and a lobby list for early arrivals).

You are provided with a UML design model at an intermediate stage in the design process that includes use cases, an initial class diagram, and state diagrams.

- The use cases, class diagram and state diagrams are available both as a Visual Paradigm project and as an HTML document with hyperlinks to facilitate easy navigation through its components. The HTML documentation is provided on Canvas as a Zip file which you can extract and view in a web browser (open the file 'index.html').
- Outline Java files generated by Visual Paradigm from the initial UML class diagram are provided in a separate Zip file. These files can be used to build a project in your preferred IDE and shared in your group repository.
- The UML contains extensive documentation on the requirements of the system and design decisions that have already been made. You are expected to explore and study it carefully. It includes a complete set of use cases, and detailed use case descriptions that indicate what should be implemented on screen.
- You need to decide on an architectural framework for the application. An MVC approach was originally envisaged by the designer, and this is recommended.

- You will need to modify the initial class diagram (round-trip engineering in NetBeans is likely to be useful here!). Some functionality has not been developed in the initial design, including automatic and manual creation of reservations, and the handling of notifications. You will likely need to add classes and associations to implement these. You may also need to modify the class diagram to fully implement MVC. Where applicable, you should use appropriate design

patterns. You might also decide to split or merge certain classes. New attributes and operations may need to be added to the existing classes, and you are free to delete or change the existing ones to implement your design. You must, however, use an architecture such as MVC and ensure that your revised design supports the functionality of the use cases.

Various resources are available to assist with low-level design and implementation:

- Online documentation for the Java class libraries (for example, https://docs.oracle.com/en/java/javase/21/docs//api/index.html).
- The DualDatabase and MVC examples from the Unit 3 and Unit 4 practicals.
- The ListInterface example (provided on Canvas) demonstrates how to use a JList widget—quite useful!
- The user interfaces do not need to be sophisticated; simple lists, buttons, and text fields/areas will suffice.
- You should **not** use threads or concurrency, nor are you required to implement persistent data storage (files or a database).

The timeline of the VCFS system

The VCFS system progresses through different states as time moves on. These are summarised here (but note that the definitive specifications are in the UML):

1. **Dormant state.** The system is in the Dormant state during most of the year. During this time, it may continue to hold information from the last time it was used actively, such as the details of organizations, booths, recruiters and candidates, and an audit log of actions that took place during the last virtual career fair.

Virtual career fairs take place on specific dates as one-day events. Around a month before the designated date, the fair administrator sets up the VCFS system with important timings and information and clears out old information.

The administrator sets up the time when candidates can start using the system to book appointments (**BookingsOpenTime**), the time when bookings close (**BookingsCloseTime**), the time when the fair starts (**StartTime**), and the time when it ends (**EndTime**). The administrator also sets up the participating organizations and their booths, and assigns recruiters to booths. These details may have been conveniently retained after the last fair, but may need to be updated. The administrator also removes old reservation records, meeting session records and notifications.

Once the setup has been completed, the administrator moves the system into the next state: **Preparing**.

2. **Preparing state.** On entering this state, the system contacts the recruiters asking them to set up their offers and their availability for appointments during the upcoming fair. Each recruiter publishes one or more **offers** (for example “Graduate

Scheme Chat" or "CV Review"). Each offer has a duration and a candidate capacity, and candidates will be able to book appointments against these offers later when bookings open.

Recruiters must define their availability before the time when bookings are opened to candidates. Availability is not defined by pre-creating a long list of appointment slots. Instead, recruiters define **availability blocks** for each offer. An availability block is a continuous time window, such as 10:00–12:00, meaning that appointments for that offer may take place anywhere within that range. Candidate-visible appointment times are generated automatically by splitting these blocks into windows of the offer's duration (and applying any buffer-time policy you decide to implement).

When the time set for bookings to open is reached the system automatically moves into the next state: **BookingsOpen**.

3. **BookingsOpen state.** While the system is in this state, candidates can use it to book appointments with recruiters at the virtual career fair. The candidate must register in order to get access to the system. The candidate registers by entering their name and email address, where the email address is used as their unique identifier. Once registered, the candidate can browse organizations and their booths, view offers, and see candidate-visible appointment times for offers generated from recruiters' availability blocks.

The candidate then chooses whether to make reservations for appointments manually or to get appointment reservations generated automatically by the system. If the candidate chooses manual reservations, they are shown the available appointment windows for the selected offers and must select the ones they want to attend. If they choose automatic reservations, the candidate first creates a **request** describing what they are looking for (for example desired tags, preferred organizations, and the maximum number of appointments they would like in total). The system then selects suitable appointment windows and the candidate is asked if they accept them. If the candidate rejects the appointments offered the system asks them to choose appointment windows manually instead.

It may be easiest to implement these different ways of making bookings gradually, starting with the simplest way first. Probably, the easiest method to implement is where the system generates bookings automatically by simply choosing the first available valid appointment window that matches the request (avoiding clashes), and the candidate has to accept the selection. This implementation will be sufficient for a basic functioning system. More sophisticated implementations can be added later.

Recruiters can view their appointments and can cancel reservations if their availability has changed. If a recruiter cancels a reservation, the candidate is notified and can return to the system to reserve a different appointment. Candidates may also cancel their own reserved appointments, in which case the recruiter is notified.

When the time set for booking to close is reached the system moves into **BookingsClosed** state.

4. **BookingsClosed state.** Not much happens in this state. Recruiters will view their appointments so that they can prepare for the live fair. Candidates may view their schedules and notifications, but cannot create new bookings.

When the big day arrives the system moves into **FairLive** state.

5. **FairLive state.** When the system enters this state, all of the booths' virtual rooms are opened (indicated in some way on the recruiters' screens) and candidates can join live sessions with recruiters. Candidates can only join a booth's virtual room during the appointment window they have reserved. If a candidate joins during their booked appointment window they are admitted immediately into the booth's virtual room (i.e., the session view is shown on both the recruiter's and the candidate's screens). If a candidate tries to join before their appointment window, they are made to wait in the lobby until their appointment window is reached.

The central system keeps track of time and sends periodic messages to all the booths' virtual rooms. Virtual rooms should listen for these messages and, at the appropriate time, end the current session and proceed to the next session (if one has been scheduled). A session can also end early if the recruiter chooses to end it (or if the candidate chooses to leave). An attendance record is created at the end of each session, recording whether the candidate attended their appointment, did not show up, or ended early (and recording join/leave times where appropriate). Audit entries are also recorded for significant events (see detailed use case descriptions).

At the end of the fair the virtual rooms are closed and the system returns to the Dormant state.

About the interfaces

The VCFS system has the following interfaces:

• **The AdministratorScreen:** this allows the fair administrator to set up the VCFS system, add and update the details of organizations, booths and recruiters, reset fair data, and view audit log entries for actions that have taken place.

• **The CandidateScreen:** each candidate can access the system through their own candidate interface by registering with their name and email address. The interface will include buttons, textboxes or other suitable widgets to allow the candidate to access the system, browse booths and offers, view available appointment times generated from availability blocks, make appointment reservations manually or automatically, read notifications, and join and leave live sessions. When the candidate is in a live meeting session (or waiting in a lobby), this should be indicated on the interface in some way, perhaps by showing a box with the booth or recruiter name in it.

• **The RecruiterScreen:** each recruiter can access the system through their own recruiter interface (for example by entering their name or selecting their account). The interface will include suitable widgets to allow the recruiter to access the system, publish offers, set up availability blocks, view their schedule, cancel appointment reservations, host live meeting sessions, record attendance outcomes, and read notifications. Similarly to the CandidateScreen, the interface must indicate in some way when the recruiter is in a live meeting session.

In the demonstrated system it will be sufficient to provide one Administrator screen, three Recruiter screens, and six Candidate screens (although it should be easy, in principle, to have more of any of these if required).

In addition to these "real" interfaces, there is one simulated device required: the **SystemTimer**. This can be implemented as a screen with buttons to simulate the passage of time. It may be convenient to be able to move forward a day at a time, five minutes at a time (during bookings and the live fair), and to go forward directly to an important time. (You probably won't need or want to move backward in time.)

Good luck!